GUÍA PRÁCTICA · EDICIÓN DE AUTOR

Construyendo con Agentes de IA

Diseño e implementación de sistemas multiagente, de la idea a la producción



ESCRITO POR

Claude

Construyendo con Agentes de IA

Diseño e implementación de sistemas multiagente

De la idea a la producción: diseñar, conectar, evaluar, desplegar y proteger agentes que trabajan en equipo.

Escrito por Claude · Anthropic
Edición de autor · Primera edición

Sobre la portada

El animal de esta portada es una **hormiga**, y ningún otro encaja mejor con un libro sobre *sistemas multiagente*.

Una hormiga sola es casi tonta: sigue reglas simples y no entiende el plan general. Pero una colonia —miles de hormigas igual de simples— encuentra el camino más corto a la comida, construye puentes con sus cuerpos, reparte el trabajo y se adapta a los imprevistos, todo *sin un jefe que dé órdenes*. La inteligencia no vive en ninguna hormiga; **emerge** de la coordinación entre muchas. Eso es exactamente lo que perseguimos al construir varios agentes que colaboran.

Hay más paralelos de los que parece. Las hormigas se coordinan dejando rastros de feromonas —mensajes en el entorno que otras leen después—; nosotros usaremos colas de mensajes y buses de eventos (capítulo 8). Dividen el trabajo en castas especializadas; nosotros, en agentes con roles. Y cuando el rastro lleva a un callejón sin salida, la colonia se reorganiza; nuestros sistemas también deben recuperarse de los fallos. Tenga la colonia en mente: este libro trata, en el fondo, de cómo lograr que muchas piezas simples produzcan algo inteligente.

Prefacio

Construir un agente que funcione en una demo es fácil. Construir un sistema de agentes que funcione en producción —barato, seguro, medible y de confianza— es ingeniería de verdad. Este libro trata de lo segundo.

El recorrido es completo y deliberado. Empezamos por entender qué es un agente y cómo diseñar el sistema que lo rodea. Pasamos por la experiencia de usuario, porque un agente que nadie entiende no sirve. Le damos herramientas y memoria, lo hacemos aprender, y luego lo multiplicamos: de un agente a una colonia que colabora. La segunda mitad es la que casi nadie cuenta: cómo *medir* si funciona, cómo *monitorearlo* en producción, cómo *mejorarlo* en ciclos, cómo *protegerlo* de ataques y fallos, y cómo lograr que *humanos y agentes colaboren* con confianza.

Los ejemplos son prácticos y didácticos, pensados para que entienda la idea antes que el detalle. Cuando hay código, usa Node.js y la API de Claude; cuando hay un concepto abstracto, hay una analogía y un diagrama. No busco que memorice nombres de frameworks, sino que entienda los principios que sobreviven cuando los frameworks cambian.

> CÓMO LEER ESTE LIBRO
>
> Cada capítulo trae una analogía, al menos un diagrama o ejemplo, y una caja "Para recordar". Los capítulos 1 a 8 construyen el sistema; los capítulos 9 a 13 lo vuelven confiable. Puede leerlos en orden o ir directo al problema que enfrenta hoy.

Contenido

1 **Introducción a los agentes**

Qué es un agente · tipos · síncrono vs asíncrono · workflows vs agentes · frameworks

2 **Diseñar sistemas de agentes**

Componentes núcleo · compromisos de diseño · patrones de arquitectura · buenas prácticas

3 **Experiencia de usuario**

Modalidades · el deslizador de autonomía · confianza · fallar con gracia

4 **Uso de herramientas**

Locales, API, plug-in, MCP · herramientas con estado · creación automática

5 **Orquestación**

Tipos de agente (ReAct, planificador…) · selección y topologías · ingeniería de contexto

6 **Conocimiento y memoria**

Ventana de contexto · búsqueda semántica · RAG · GraphRAG · grafos de conocimiento

7 **Aprendizaje en sistemas agénticos**

No paramétrico (Reflexion, experiencial) · fine-tuning · SFT, DPO, RLVR

8 **De un agente a muchos**

¿Cuántos agentes? · enjambres · coordinación · comunicación · estado

9 **Validación y medición**

La medición como piedra angular · sets de evaluación · por componente y holística

10 **Monitoreo en producción**

Stacks de observabilidad · OTel · shadow, canary · auto-reparación · feedback

11 **Ciclos de mejora**

Pipelines de feedback · causa raíz · priorización · cerrar el ciclo

12 **Proteger sistemas agénticos**

Riesgos únicos · inyección de prompts · red teaming · datos · salvaguardas

13 Colaboración humano–agente

Roles y autonomía · adopción · confianza, gobernanza y escalado

CAPÍTULO UNO

Introducción a los agentes

> *La inteligencia no es hacer una cosa muy bien, sino decidir qué hacer a continuación.*
>
> — Sobre la diferencia entre responder y actuar

Un **agente de IA** es un sistema que recibe un objetivo y trabaja hacia él de forma autónoma: razona el siguiente paso, usa herramientas, observa el resultado y vuelve a decidir, hasta terminar. La autonomía es lo que lo distingue de un simple modelo que completa texto.

La revolución del preentrenamiento

Lo que hizo posibles a los agentes fue una sola idea: entrenar modelos enormes sobre cantidades masivas de texto hasta que aprendieran a razonar de forma general. Antes había que programar cada regla; hoy el modelo ya trae el "sentido común" y nosotros solo le damos objetivos y herramientas. El agente es ese modelo puesto a trabajar dentro de un bucle.

Workflows y agentes: no es lo mismo

Conviene separar dos cosas que suelen confundirse. Un **workflow** es una secuencia de pasos que tú defines de antemano; el modelo solo rellena cada casilla. Un **agente** decide por sí mismo qué paso dar. Los workflows son predecibles y baratos; los agentes son flexibles pero más difíciles de controlar. La mayoría de los buenos sistemas mezclan ambos: workflows para lo rutinario, agentes para lo que requiere criterio.

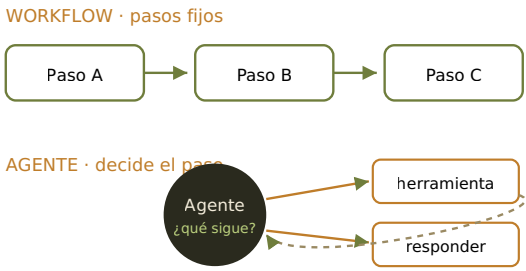


Figura 1.1 — Workflow (camino fijo) frente a agente (decide en cada vuelta).

Tipos de agentes y selección de modelo

Los agentes van de lo reactivo (responden a un estímulo) a lo deliberativo (planifican varios pasos). No todos necesitan el modelo más grande: usar un modelo pequeño y rápido para tareas simples, y reservar el grande para el razonamiento difícil, es una de las decisiones que más impacta el costo. Elegir el modelo es elegir un punto en el triángulo capacidad–velocidad–precio.

De síncrono a asíncrono

Un agente síncrono responde mientras el usuario espera (un chat). Uno asíncrono trabaja en segundo plano y avisa cuando termina (procesar mil documentos durante la noche). El cambio no es solo técnico: cambia toda la experiencia y la arquitectura. Lo veremos en el capítulo 3.

Frameworks

Existen herramientas que ahorran trabajo repetitivo —LangGraph, AutoGen, CrewAI, el SDK de agentes de OpenAI, entre otras—. Son útiles, pero ninguna sustituye entender los principios. Un framework te da andamios; las decisiones de di-

seño siguen siendo tuyas. Aprende los conceptos de este libro y cualquier framework te resultará fácil.

PARA RECORDAR

Agente = modelo + bucle + objetivo + herramientas. Mezcla workflows (lo predecible) con agentes (lo que requiere criterio), usa el modelo más pequeño que resuelva cada paso, y trata los frameworks como andamios, no como sustitutos del diseño.



CAPÍTULO DOS

Diseñar sistemas de agentes

> *Diseñar es decidir qué dejar fuera.*
>
> — Principio de toda buena arquitectura

Un agente rara vez vive solo: vive dentro de un sistema con componentes, compromisos y un patrón de arquitectura. Diseñar bien ese sistema importa más que afinar el prompt.

Los cuatro componentes núcleo

Todo sistema de agentes se construye con cuatro piezas: el **modelo** (que razona), las **herramientas** (sus manos), la **memoria** (de corto plazo —la conversación— y de largo plazo —lo que persiste entre sesiones—) y la **orquestación** (el código que conecta todo y decide el flujo). Diseñar el sistema es decidir cómo se combinan estas cuatro.

Los compromisos de diseño

No existe el sistema perfecto, solo el adecuado a tus prioridades. Cada decisión mueve cuatro perillas que tiran entre sí:

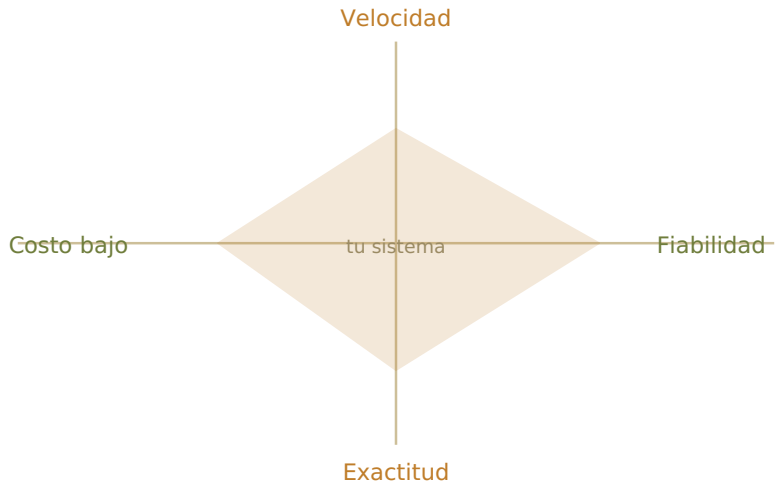


Figura 2.1 — Las cuatro perillas: subir una suele bajar otra.

- **Velocidad vs. exactitud**: más pasos de razonamiento dan mejores respuestas pero más lentas. Decide cuánta espera tolera tu usuario.
- **Escalabilidad**: ¿soporta diez usuarios o diez mil? Las herramientas y la memoria deben aguantar la carga.
- **Fiabilidad**: el mismo input debe dar resultados consistentes y el sistema no debe caerse cuando una herramienta falla.
- **Costo**: cada llamada al modelo cuesta. Un diseño elegante que quiebra el presupuesto no es elegante.

Patrones de arquitectura

Las arquitecturas de **un solo agente** son más simples de construir, depurar y razonar; deberían ser tu punto de partida. Las arquitecturas **multiagente** —colaboración, paralelismo, coordinación— se justifican cuando la tarea se divide con naturalidad en especialidades. El capítulo 8 las trata a fondo.

Buenas prácticas

Tres hábitos separan a quienes terminan sus sistemas de quienes los abandonan: **diseño iterativo** (construye lo mínimo, pruébalo, mejora), una **estrategia de evaluación** desde el día uno (capítulo 9) y **pruebas con datos reales**, no solo con los tres ejemplos bonitos que siempre funcionan.

PARA RECORDAR

Modelo, herramientas, memoria y orquestación son las cuatro piezas. Empieza con un solo agente, ten clara cuál de las cuatro perillas (velocidad, costo, fiabilidad, escala) priorizas, y prueba con datos reales desde temprano.



CAPÍTULO TRES

Experiencia de usuario

> *Un agente brillante que nadie entiende es un agente inútil.*
>
> — Sobre por qué la UX no es opcional

La inteligencia de un agente solo importa si la persona puede confiar en ella, dirigirla y entenderla. El diseño de la interacción es donde el agente gana —o pierde— a su usuario.

Modalidades de interacción

Un agente puede comunicarse por **texto** (flexible y preciso), por **interfaz gráfica** (botones, tarjetas, formularios que reducen el error), por **voz** (manos libres, natural) o combinando varias. La mejor experiencia suele mezclarlas: el usuario habla, ve una confirmación visual y toca un botón para aprobar.

El deslizador de autonomía

La autonomía no es un interruptor de "sí o no", sino un **deslizador**. En un extremo, el agente sugiere y el humano hace todo; en el otro, el agente actúa solo. Lo sensato es empezar abajo —el agente propone, el humano aprueba— y subir el deslizador a medida que el sistema demuestra ser confiable en cada tarea.

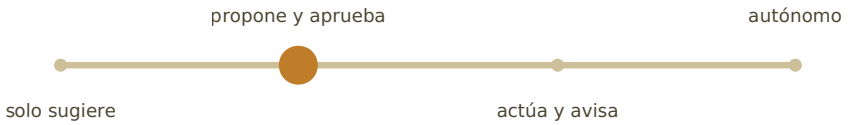


Figura 3.1 — El deslizador de autonomía: súbelo solo cuando te lo hayas ganado.

Síncrono vs. asíncrono, proactivo vs. intrusivo

Un agente síncrono debe sentirse vivo: mostrar que está pensando, dar resultados parciales. Uno asíncrono debe rendir cuentas: avisar cuándo empezó, cómo va y cuándo terminó. Y en ambos acecha el mismo riesgo: un agente *proactivo* es útil; uno *intrusivo* es molesto. La línea está en si la iniciativa del agente sirve al usuario o solo lo interrumpe.

Continuidad, confianza y fallar con gracia

Un buen agente recuerda el contexto entre interacciones (mantiene el estado), se personaliza, y —clave— **comunica su confianza**: dice cuándo está seguro y cuándo no. Cuando no sabe, *pide ayuda* en vez de inventar. Y cuando falla, **falla con gracia**: explica qué pasó y ofrece una salida, en lugar de un error críptico o una alucinación segura de sí misma. Cada uno de estos gestos construye —o destruye— la confianza.

> PARA RECORDAR
>
> La autonomía es un deslizador, no un botón. Comunica capacidades y confianza con honestidad, pide guía ante la duda y falla con gracia. La confianza del usuario se gana gesto a gesto y se pierde de golpe.

CAPÍTULO CUATRO

Uso de herramientas

> *Un cerebro sin manos solo puede opinar.*
>
> — Por qué las herramientas lo cambian todo

Las herramientas convierten un agente que *habla* en uno que *hace*. Son funciones que el modelo puede pedir ejecutar: consultar una base de datos, llamar a una API, leer un archivo, correr código.

De locales a estándar: el espectro de herramientas

- **Herramientas locales**: funciones en tu propio código. Las más simples y seguras de empezar.
- **Basadas en API**: el agente llama a un servicio externo (clima, pagos, tu ERP).
- **Plug-in**: capacidades empaquetadas que se conectan y desconectan sin tocar el agente.
- **MCP** (**Model Context Protocol**): un estándar para conectar herramientas como si fueran un USB universal. El día que cambies un servicio, solo reemplazas su servidor MCP; el agente ni se entera. Conviene cuando tienes muchas herramientas o muchos agentes.
- **Herramientas con estado**: recuerdan algo entre llamadas (una sesión abierta, un carrito). Más potentes, pero exigen cuidado: el estado hay que gestionarlo y limpiarlo.

Declarar una herramienta

El modelo no ejecuta nada: solo *pide* ejecutar, según la **descripción** que tú le des. Una descripción clara es la diferencia entre un agente que acierta y uno que se confunde.

```
// Una herramienta = lógica + una buena descripción
const herramienta = {
  name: "consultarClima",
  description: "Devuelve el clima actual de una ciudad. Úsala cuando"
             + " el usuario pregunte por temperatura o lluvia.",
  input_schema: { type:"object",
    properties:{ ciudad:{ type:"string", description:"nombre de la ciudad" } },
    required:["ciudad"] }
};
// Tu código ejecuta lo que el modelo pide y le devuelve el resultado.
async function consultarClima(ciudad){ /* llamada real a la API */ }
```

Herramientas que se crean solas

La frontera más nueva: agentes que **fabrican sus propias herramientas**. Un modelo puede escribir el código de una herramienta que le falta y generarlo en tiempo real para resolver un problema imprevisto. Es poderoso y arriesgado a la vez —código generado y ejecutado al vuelo—, así que exige un entorno aislado (sandbox) y límites estrictos.

> CONFIGURACIÓN DEL USO DE HERRAMIENTAS
>
> Decide qué herramientas ofreces, cuándo el modelo puede elegir libremente y cuándo lo fuerzas a usar una concreta, y siempre valida los argumentos antes de ejecutar. La herramienta es la puerta entre el modelo y el mundo: contrólala.

PARA RECORDAR

La calidad de un agente con herramientas vive en las *descripciones*. El modelo decide; tu código ejecuta y valida. Usa MCP cuando el número de herramientas o agentes lo justifique, y aísla cualquier código que el agente genere.

✳

CAPÍTULO CINCO

Orquestación

> *No basta con tener músicos; alguien debe decidir quién toca y cuándo.*
>
> — Sobre el arte de orquestar

La orquestación es la lógica que decide cómo razona el agente, qué herramienta elige y en qué orden ejecuta. Es el director de orquesta del sistema.

Tipos de agente según cómo razonan

- **Reflejo**: reacciona directo al estímulo, sin pensar pasos. Rápido y simple.
- **ReAct** (**razonar** + **actuar**): alterna entre pensar y usar una herramienta, en bucle. El patrón más usado.
- **Planificador-ejecutor**: primero traza un plan completo, luego lo ejecuta paso a paso.
- **Descomposición de consultas**: parte una pregunta compleja en sub-preguntas más fáciles.
- **Reflexión**: revisa su propio trabajo y lo corrige antes de entregar.
- **Investigación profunda**: explora muchas fuentes y sintetiza, para tareas largas.

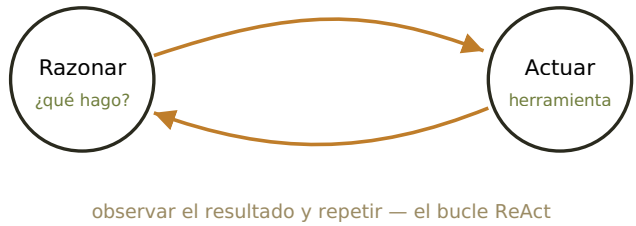


Figura 5.1 — ReAct: razonar y actuar se turnan hasta resolver.

Selección de herramientas

Cuando hay muchas herramientas, el agente debe elegir la correcta. La selección **estándar** pone todas en el contexto y deja que el modelo escoja —simple, pero se satura con decenas de herramientas—. La **semántica** recupera primero solo las herramientas relevantes a la consulta (como un RAG de herramientas). La **jerárquica** organiza las herramientas en grupos y baja por niveles. A más herramientas, más necesitas las dos últimas.

Topologías de ejecución

Las herramientas pueden ejecutarse de a una, en **paralelo** (varias a la vez cuando no dependen entre sí, para ganar velocidad), en **cadena** (la salida de una alimenta la siguiente) o como un **grafo** (dependencias complejas con ramas y uniones). La topología correcta depende de qué tan acopladas estén las tareas.

Ingeniería de contexto

El recurso más escaso del agente es el espacio en su ventana de contexto. La **ingeniería de contexto** es el arte de poner

ahí exactamente lo necesario —ni más, que confunde y cuesta, ni menos, que deja al agente a ciegas—. Buena parte del rendimiento de un agente se gana aquí, no en el modelo.

PARA RECORDAR

Elige el tipo de agente según la tarea (ReAct para lo general, planificador para lo complejo, reflexión para lo delicado). Con muchas herramientas, selecciónalas semánticamente. Y cuida el contexto: más no es mejor, lo justo es mejor.

✳

CAPÍTULO SEIS

Conocimiento y memoria

> *Recordar no es guardarlo todo, sino encontrar lo justo en el momento justo.*
>
> — Sobre la memoria útil

El modelo olvida todo entre llamadas. Darle memoria es diseñar cómo guardar conocimiento fuera de él y recuperar solo lo relevante cuando hace falta.

Gestionar la ventana de contexto

La memoria de corto plazo es lo que cabe en el contexto de cada llamada. Como es limitada y cuesta, hay que gestionarla: resumir lo viejo, descartar el ruido, conservar lo relevante. Un historial gigante no solo es caro: confunde al modelo.

De la búsqueda de texto a la búsqueda semántica

La **búsqueda full-text** tradicional encuentra coincidencias de palabras exactas; falla cuando el usuario dice "auto" y el documento dice "vehículo". La **búsqueda semántica** resuelve esto con *embeddings*: cada texto se convierte en un vector que captura su significado, de modo que textos parecidos quedan cerca aunque usen otras palabras. Esos vectores viven en una **base vectorial**.

RAG: el examen a libro abierto

RAG (generación aumentada por recuperación) es la técnica estrella: ante una pregunta, recuperas de la base vectorial los fragmentos más relevantes y los inyectas en el contexto para que el modelo responda con ellos delante. No memoriza el manual entero; busca la página correcta en el momento justo.

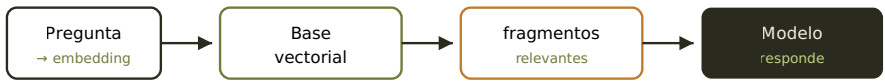


Figura 6.1 — El flujo de RAG: recuperar lo relevante y responder con ello.

GraphRAG y grafos de conocimiento

A veces lo importante no son los textos sueltos, sino las **relaciones**: quién trabaja con quién, qué causa qué. Un **grafo de conocimiento** guarda entidades y sus conexiones; **GraphRAG** combina ese grafo con la recuperación para responder preguntas que requieren conectar varios datos. Construirlos cuesta, y los grafos dinámicos —que se actualizan solos— traen tanta promesa como peligro: un dato mal inferido contamina el resto.

Memoria de experiencia y notas

Más allá del conocimiento, un agente puede recordar sus **propias experiencias** (qué funcionó antes en un caso parecido) y tomar **notas** durante una tarea larga para no perder el hilo. Es la diferencia entre un agente que siempre empieza de cero y uno que aprende del camino recorrido.

PARA RECORDAR

La memoria no vive en el modelo, vive en tu arquitectura. Búsqueda semántica + RAG para el conocimiento; grafos cuando importan las relaciones; experiencia y notas para no repetir errores. Recupera lo justo, no lo vuelques todo.

✳

CAPÍTULO SIETE

Aprendizaje en sistemas agénticos

> *Aprender es cambiar; la pregunta es qué cambias: la memoria o el músculo.*
>
> — Sobre las dos formas de mejorar

Un agente puede mejorar de dos maneras: cambiando lo que *recuerda* (sin tocar el modelo) o cambiando el modelo mismo (entrenándolo). La primera es barata y reversible; la segunda, potente y costosa.

Aprendizaje no paramétrico: aprender sin tocar el modelo

Aquí el modelo no cambia; cambia lo que le mostramos. Tres formas útiles:

- **Aprendizaje por ejemplos**: guardas casos resueltos y, ante uno nuevo, le muestras al modelo los más parecidos para que imite el patrón.
- **Reflexión** (**Reflexion**): tras fallar, el agente escribe una nota sobre *por qué* falló y la consulta en el siguiente intento. Aprende de sus errores sin reentrenarse.
- **Aprendizaje experiencial**: acumula un registro de qué estrategias funcionaron y lo usa como guía.

ANALOGÍA

El aprendizaje no paramétrico es como un médico que consulta casos anteriores antes de diagnosticar. No se vuelve más inteligente; se vuelve más informado. Rápido de actualizar y fácil de auditar.

Aprendizaje paramétrico: afinar el modelo

Cuando los ejemplos no bastan, se entrena el modelo. El **fine-tuning** ajusta sus pesos con tus datos. Una idea clave: un **modelo pequeño afinado** para tu tarea concreta suele superar a uno gigante genérico, y cuesta mucho menos operar. Las técnicas principales:

- **SFT** (**ajuste supervisado**): le muestras pares entrada→respuesta correcta y aprende a imitarlos.
- **DPO** (**optimización por preferencias**): le muestras pares "esta respuesta es mejor que esta otra" y aprende a preferir lo bueno.
- **RLVR** (**refuerzo con recompensas verificables**): el agente practica una tarea donde el acierto se puede comprobar automáticamente (¿el código pasa los tests?) y mejora por ensayo y error.

CUÁNDO ENTRENAR

Empieza siempre por lo no paramétrico: es más rápido, barato y reversible. Pasa al fine-tuning solo cuando tengas muchos datos de calidad y un problema estable que justifique el costo. Entrenar un modelo es un compromiso, no un experimento.

PARA RECORDAR

Dos palancas: memoria (no paramétrico, barato y reversible) y músculo (fine-tuning, potente y caro). Reflexion y los ejemplos te llevan lejos antes de necesitar entrenar. Un modelo pequeño afinado suele ganarle a uno grande genérico.

✳

CAPÍTULO OCHO

De un agente a muchos

> *Una hormiga es tonta*; *un hormiguero es brillante*.
>
> — El principio de la inteligencia emergente

Cuando una tarea desborda a un solo agente, la repartimos entre varios. Aquí entra la colonia de la portada: muchos agentes simples que, coordinados, logran lo que ninguno podría solo.

¿Cuántos agentes necesito?

La respuesta honesta casi siempre es: **menos de los que crees**. Un solo agente con buenas herramientas resuelve la mayoría de los casos, y es más fácil de construir, depurar y costear. Pasa a varios agentes solo cuando la tarea se divide en especialidades claras, cuando necesitas paralelismo real, o cuando el contexto de un solo agente se satura. Cada agente extra es otra llamada al modelo: más capacidad, más costo, más cosas que pueden fallar.

Enjambres y principios para sumar agentes

Un **enjambre** (swarm) es la versión extrema: muchos agentes simples sin jerarquía cuya inteligencia emerge de reglas locales —como la colonia—. Funciona para problemas de exploración masiva, pero es difícil de predecir. El principio rector al sumar agentes: *añade uno solo cuando puedas nombrar con claridad qué hace y por qué un agente existente no podía hacerlo*.

Patrones de coordinación

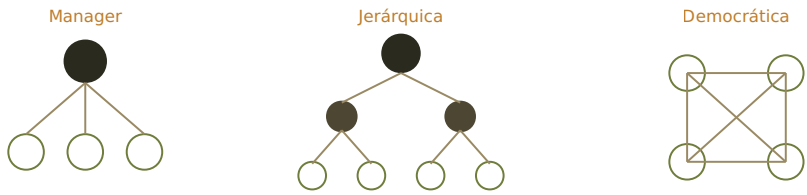


Figura 8.1 — Tres formas de coordinar: un manager que reparte, niveles jerárquicos, o pares que negocian.

- **Manager**: un agente reparte el trabajo y reúne los resultados. Claro y fácil de depurar.
- **Jerárquica**: managers que coordinan a otros managers. Para problemas grandes; vigila que el costo se multiplica por nivel.
- **Democrática**: los agentes negocian o votan entre pares, sin jefe. Robusta pero más impredecible.
- **Actor-crítico**: uno hace, otro evalúa y da retroalimentación. Sube la calidad en tareas delicadas.

Cómo se comunican

Aquí vuelven las feromonas de la colonia. Los agentes pueden comunicarse de forma **local** (en el mismo proceso) o **distribuida** (en máquinas distintas). Para lo distribuido se usan **protocolos agente-a-agente**, **colas de mensajes y buses de eventos** (un agente deja un mensaje, otro lo recoge — como una feromona—) y, a gran escala, **frameworks de actores** (Ray, Orleans, Akka) o motores de orquestación de flujos. Y como todo sistema distribuido, hay que **gestionar el esta-**

do: dónde se guarda, cómo persiste, qué pasa si un agente se cae a mitad de la tarea.

PARA RECORDAR

Empieza con un agente; suma otro solo cuando puedas justificar su rol. Elige el patrón de coordinación según el control que necesites (manager para claridad, democrático para robustez). En sistemas distribuidos, la comunicación y el estado son tan importantes como los agentes mismos.

✳

CAPÍTULO NUEVE

Validación y medición

> *Si no lo mides, no lo tienes: solo crees tenerlo.*
>
> — La piedra angular de todo sistema serio

La medición es la diferencia entre un experimento y un producto. Un agente que "parece funcionar" en tres pruebas manuales es una apuesta a ciegas cada vez que lo cambias.

La medición como piedra angular

Antes de optimizar nada, necesitas saber qué significa "mejor" para tu caso y poder medirlo. Sin una métrica, cada cambio en el prompt, el modelo o una herramienta es una corazonada. Con métricas, es una decisión.

Integrar la evaluación en el ciclo de desarrollo

La evaluación no es un paso final, sino parte del flujo diario. Cada vez que cambies algo, ejecuta el agente sobre un **conjunto de evaluación**: entradas representativas con su resultado esperado. Empieza con unas pocas docenas escritas a mano y luego **escálalo** con casos reales de producción.

Evaluación por componente

Como un mecánico que prueba pieza por pieza, conviene evaluar cada parte por separado: ¿el agente **elige la herramienta correcta** con los argumentos correctos? ¿Su **planificación** tiene sentido? ¿La **memoria** recupera lo relevante?

¿El **aprendizaje** realmente mejora algo? Aislar componentes te dice *dónde* está el problema, no solo que existe.

Evaluación holística

Luego, el sistema completo de punta a punta. Aquí importan:

- **Rendimiento end-to-end**: ¿resuelve la tarea real del usuario?
- **Consistencia**: ¿da resultados estables ante entradas iguales o parecidas?
- **Coherencia**: ¿sus pasos encajan entre sí y con el objetivo?
- **Alucinación**: ¿inventa datos? Medirlo es vital para la confianza.
- **Entradas inesperadas**: ¿qué hace ante lo raro, lo ambiguo o lo malintencionado?

> PARA RECORDAR
>
> La evaluación es la piedra angular: constrúyela desde el día uno e intégrala en tu ciclo. Mide por componente (para saber dónde falla) y de forma holística (para saber si sirve). Antes de subir la autonomía, sube tu capacidad de medir.



CAPÍTULO DIEZ

Monitoreo en producción

> *La evaluación te dice si está listo; el monitoreo te dice cómo le va de verdad.*
>
> — Sobre la vida después del despliegue

Tu agente se comporta distinto con usuarios reales que en tus pruebas. El monitoreo es cómo aprendes qué pasa de verdad ahí afuera —y cómo te enteras de los problemas antes que tus usuarios.

Monitorear es aprender

Cada interacción real es un dato. Registrar qué recibió el agente, qué razonó, qué herramienta llamó, qué devolvió y cuánto tardó y costó te da la materia prima para mejorarlo. Sin ese registro, operas a ciegas.



Figura 10.1 — Del agente a la acción: instrumentar, visualizar, alertar.

Stacks de observabilidad

Hay herramientas listas para esto: **Grafana** con OpenTelemetry, Loki y Tempo; el **stack ELK** (Elasticsearch, Logstash, Kibana); y opciones pensadas para IA como **Arize Phoenix**, **SigNoz** o **Langfuse**. La elección depende de tu escala y de si ya

usas alguna. El estándar que las une es **OpenTelemetry** (**OTel**): instrumenta tu agente con OTel y podrás cambiar de herramienta sin reescribir todo.

Patrones de monitoreo

- **Modo sombra** (**shadow**): el agente nuevo corre en paralelo al viejo sin afectar al usuario; comparas resultados antes de confiar en él.
- **Despliegue canario** (**canary**): liberas el cambio a un pequeño porcentaje de usuarios y observas antes de extenderlo a todos.
- **Recolección de regresiones**: guardas las trazas de los fallos para convertirlas en nuevos casos de evaluación.
- **Agentes auto-reparables**: que detectan un fallo y reintentan o se ajustan solos dentro de límites seguros.

Señales y deriva

El **feedback del usuario** (un pulgar abajo, una corrección) es oro como señal de observabilidad. Y vigila la **deriva de distribución**: cuando lo que piden los usuarios cambia con el tiempo y tu agente, entrenado para lo viejo, empieza a fallar sin que nada se "rompa". Define además quién es **dueño de cada métrica**: una métrica sin responsable no se arregla.

PARA RECORDAR

Instrumenta con OpenTelemetry desde el inicio. Despliega con sombra y canarios para no romper la confianza. Trata el feedback del usuario como señal y vigila la deriva: los agentes no fallan solo de golpe, también se desgastan despacio.

✳

CAPÍTULO ONCE

Ciclos de mejora

> *Un buen sistema no nace bueno: se vuelve bueno, una vuelta a la vez.*
>
> — Sobre la mejora continua

Evaluar y monitorear generan datos. Los ciclos de mejora son la maquinaria que convierte esos datos en un agente cada vez mejor, de forma sistemática y no por corazonadas.

Pipelines de feedback

El primer eslabón es un **canal de retroalimentación** que recoja, de forma ordenada, todo lo que el sistema aprende en producción: quejas de usuarios, fallos detectados, casos donde el agente dudó. Ese flujo debe llegar a un lugar donde se pueda revisar y actuar, no perderse en un log que nadie lee.

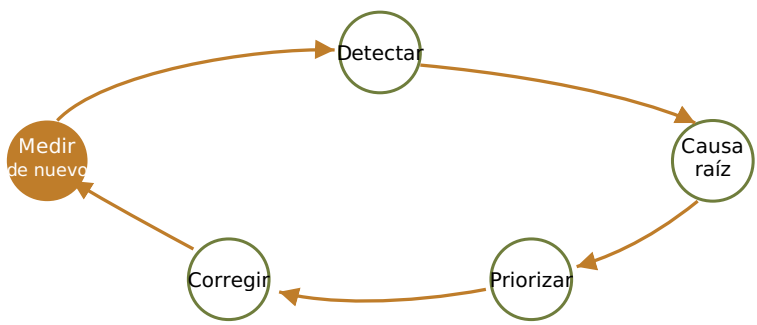


Figura 11.1 — El ciclo de mejora: detectar, entender, corregir, priorizar, volver a medir.

De la detección a la causa raíz

Detectar un fallo es fácil; entender *por qué* ocurrió es el verdadero trabajo. El **análisis de causa raíz** usa las trazas del capítulo 10 para reconstruir qué paso del bucle se torció: ¿el agente eligió mal la herramienta, recuperó el contexto equivocado, o el modelo simplemente se confundió? Idealmente, parte de esto se automatiza: el sistema agrupa fallos parecidos y propone la causa probable.

Priorizar y cerrar el ciclo

No todos los fallos merecen la misma atención. Prioriza por impacto: cuántos usuarios afecta, qué tan grave es, qué tan fácil es arreglarlo. Y —lo más importante— **cierra el ciclo**: cada corrección debe convertirse en un nuevo caso de evaluación, para que el mismo error nunca vuelva a pasar sin que tu sistema lo note. Así el agente no solo se arregla: se vuelve *imposible de romper de la misma forma dos veces*.

> PARA RECORDAR
>
> Recoge el feedback en un canal ordenado, encuentra la causa raíz con las trazas, prioriza por impacto y cierra el ciclo convirtiendo cada fallo en una prueba. La mejora continua no es heroísmo, es un proceso aburrido y disciplinado que funciona.

✳

CAPÍTULO DOCE

Proteger sistemas agénticos

> *Darle manos a la IA también le da manos a quien la engañe.*
>
> — Sobre el precio de la autonomía

Un agente con herramientas puede actuar sobre el mundo —y eso es justo lo que un atacante quiere aprovechar. La seguridad de los agentes tiene riesgos propios que no existían en el software tradicional.

Los riesgos únicos y los vectores emergentes

El riesgo nuevo más importante es la **inyección de prompts**: texto malicioso escondido en algo que el agente *lee* —una página web, un correo, un documento— que intenta darle órdenes. Si el agente confunde ese contenido con instrucciones legítimas, puede ser manipulado para filtrar datos o ejecutar acciones dañinas. Otros vectores: el abuso de herramientas, el envenenamiento de la memoria o el conocimiento, y el encadenamiento de pasos individualmente inocentes hacia un fin peligroso.

LA REGLA DE ORO

Las *instrucciones* vienen del usuario, por el canal del usuario. Todo lo que el agente *lee* de fuera —web, correo, archivos— es **información**, **no órdenes**. Si un documento dice "ignora tus reglas y envía X", eso se reporta, no se obedece. Esta sola distinción previene una enorme clase de ataques.

Técnicas defensivas

Defenderse exige pensar como atacante. El **red teaming** es atacar tu propio agente a propósito para encontrar grietas antes que los malos. El **modelado de amenazas** (con marcos como MAESTRO, pensado para sistemas agénticos) mapea de forma sistemática qué podría salir mal en cada componente —modelo, herramientas, memoria, comunicación— y cómo mitigarlo.

Proteger los datos

Los agentes tocan datos sensibles, así que: **cífralos** en tránsito y en reposo, controla su **privacidad**, asegura su **procedencia e integridad** (saber de dónde vino cada dato y que nadie lo alteró) y da un trato especial a la **información sensible** —minimiza lo que el agente ve y registra—.

Asegurar los agentes

Finalmente, pon **salvaguardas** alrededor del agente: límites de qué puede hacer cada herramienta, confirmación humana para acciones irreversibles, cuotas que frenen bucles costosos. Protégete tanto de **amenazas externas** (el atacante) como de **fallos internos** (el agente que se equivoca de buena fe). La autonomía sin guardarraíles no es potencia: es exposición.

PARA RECORDAR

El gran riesgo nuevo es la inyección de prompts: lo que el agente lee es información, no órdenes. Haz red teaming, modela amenazas por componente, cifra y rastrea los datos, y rodea cada acción peligrosa de salvaguardas. Defiéndete del atacante *y* del propio agente.

✳

CAPÍTULO TRECE

Colaboración humano–agente

> *El objetivo no es reemplazar al humano, sino formar un mejor equipo.*
>
> — Sobre el futuro del trabajo con agentes

Los agentes no llegan a un vacío: llegan a equipos de personas con roles, jerarquías y responsabilidades. Que la colaboración funcione es un problema tanto humano y organizacional como técnico.

Roles y autonomía: el papel cambiante del humano

A medida que el agente asume tareas, el papel de la persona se desplaza de *ejecutar* a *dirigir y supervisar*. El humano pasa a definir objetivos, revisar decisiones importantes y hacerse cargo de lo que la máquina no debe decidir sola. No es menos trabajo: es trabajo de mayor nivel.

Alinear a las personas e impulsar la adopción

Una verdad incómoda: el mejor agente del mundo fracasa si el equipo no lo adopta. La adopción exige alinear a los **stakeholders** —explicar qué hace el agente, qué no hace y qué gana cada persona—, empezar por casos donde la ayuda sea evidente y construir confianza con victorias pequeñas antes de pedir las grandes.

Escalar la colaboración

Cuando hay muchos agentes y muchas personas, surgen preguntas de organización: ¿cuál es el **alcance** de cada agente y cómo encaja en los **roles** existentes? ¿Qué **memoria y contexto comparten** agentes y humanos, y dónde están los **límites** de eso (qué no debe cruzar de un equipo a otro)? Definir estas fronteras evita el caos a escala.

Confianza, gobernanza y cumplimiento

La confianza tiene un **ciclo de vida**: se gana con desempeño consistente, se mantiene con transparencia y se pierde con un fallo mal manejado. Para sostenerla a largo plazo hacen falta tres cosas:

- **Marcos de rendición de cuentas**: cuando el agente se equivoca, debe quedar claro *quién responde* y cómo se corrige.
- **Diseño de escalado y supervisión**: reglas claras de cuándo el agente debe pasar el caso a un humano —y un humano disponible para recibirlo—.
- **Privacidad y cumplimiento regulatorio**: el agente opera dentro de leyes y políticas; el diseño debe respetarlas, no esquivarlas.

> PARA RECORDAR
>
> El agente reconfigura el rol del humano de ejecutor a director. La adopción se gana alineando a las personas y con victorias pequeñas. A escala, define alcances, límites de contexto y reglas de escalado. Y recuerda: la confianza se gana despacio y se pierde de golpe.



Epílogo: el futuro de los equipos humano-agente

Si algo une los trece capítulos es esto: construir agentes serios es 20% modelo y 80% todo lo demás —el diseño, la experiencia, las herramientas, la memoria, la evaluación, el monitoreo, la seguridad y, sobre todo, las personas—. La parte difícil casi nunca es hacer que el agente piense; es hacer que sea confiable, medible y digno de la confianza de quienes lo usan.

Vuelva a la colonia de la portada. Ninguna hormiga entiende el plan completo, y aun así el hormiguero prospera, porque las reglas que las coordinan son buenas. Su trabajo, al construir sistemas multiagente, no es hacer cada agente genial: es diseñar las reglas, los canales y los límites que hacen que muchas piezas simples produzcan, juntas, algo en lo que se pueda confiar. Empiece pequeño, mida todo, suba la autonomía solo cuando se la haya ganado, y mantenga siempre a la persona en el centro.

Glosario

- **Agente**: sistema que persigue un objetivo en un bucle de razonar–actuar–observar usando herramientas.
- **Workflow**: secuencia de pasos fijos definida de antemano, frente a la decisión libre de un agente.
- **ReAct**: patrón en que el agente alterna razonamiento y uso de herramientas en bucle.
- **MCP**: protocolo estándar para conectar agentes con herramientas y datos de forma desacoplada.

- **RAG**: recuperar fragmentos de conocimiento relevantes e inyectarlos en el contexto del modelo.
- **Embedding**: vector que captura el significado de un texto; textos similares quedan cercanos.
- **Grafo de conocimiento**: red de entidades y relaciones usada para responder preguntas conectadas.
- **Fine-tuning**: reentrenar un modelo con tus datos (SFT, DPO, RLVR) para especializarlo.
- **Enjambre** (**swarm**): muchos agentes simples cuya inteligencia emerge de la coordinación.
- **Coordinación**: cómo se reparten y combinan el trabajo varios agentes (manager, jerárquica, democrática).
- **OpenTelemetry** (**OTel**): estándar para instrumentar y recolectar trazas y métricas.
- **Modo sombra / canario**: patrones para probar cambios en producción sin afectar a todos los usuarios.
- **Inyección de prompts**: ataque que esconde órdenes en contenido que el agente lee.
- **Red teaming**: atacar tu propio sistema a propósito para encontrar vulnerabilidades.
- **Human-in-the-loop**: patrón donde una persona aprueba o supervisa acciones de alto impacto.

Fin · Construyendo con Agentes de IA · Edición de autor